AUTOMATED RECORD-KEEPING AND FINANCIAL MANAGEMENT SYSTEM FOR BODA-BODA GROUPS.

SYSTEM DOCUMENTATION

SUBMITTED BY,

MARYANNE NJOKI NJUE

DIT-01-0151/2025

A SYSTEM DOCUMENTATION SUBMITTED IN PARTIAL FULFILMENT FOR THE AWARD OF DIPLOMA IN INFORMATION TECHNOLOGY BY ZETECH UNIVERSITY

APRIL 2026

DECLARATION

I declare that this project, titled **"Automated Record-keeping and Financial Management System for Boda-Boda-Boda Groups"** is my original work and has not been presented for the award of any degree, diploma, or certificate in any other institution. All sources of information used in this project have been duly acknowledged.

Student Name: Maryanne Njoki Njue
Registration Number: DIT-01-0151/2025
Signature:
Date:

Supervisor's Name:
Signature:
Date:

DEDICATION

This project is dedicated to my family for their continuous support, encouragement, and patience throughout my studies. I also dedicate this work to all boda-boda operators whose daily financial challenges inspired the development of this system.

ABSTRACT

The Boda-Boda Financial Management System is a web-based application developed to help boda-boda operators manage their daily financial transactions efficiently. Many riders rely on manual record-keeping, which often leads to poor tracking of income, expenses, and payments. This system provides a digital platform where riders can register, log in, record payments, and view financial summaries, while administrators can monitor records and generate reports.

The system was developed using web technologies for the frontend, **Node.js** for the backend, and **MongoDB** for data storage. It also considers integration with mobile money services such as **M-Pesa** to reflect real-world payment methods used by riders.

The project demonstrates how information systems can improve financial accountability, transparency, and record-keeping for small-scale transport operators. The system achieved its objectives by providing an accessible and practical solution to financial management challenges in the boda-boda sector.

DEFINITION OF KEY TERMS

- **Boda-Boda:** A motorcycle taxi used for public transport.
- **Financial Management:** The process of planning, organizing, and monitoring financial activities.
- **Web Application:** A software application that runs on a web browser.
- **Database:** An organized collection of data stored electronically.
- **Authentication:** The process of verifying a user's identity.
- **Administrator (Admin):** A user with special privileges to manage system data and reports.
- **Transaction:** A record of a financial activity such as a payment.
- **Report:** A summary of financial data generated by the system.

ABBREVIATIONS AND ACRONYMS

DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
DB	Database
UI	User Interface
API	Application Programming Interface
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
CRUD	Create, Read, Update, Delete
OTP	One-Time Password

LIST OF FIGURES

Figure 2.2.2 Login Page Design

Figure 2.2.3 Dashboard Page Design

Figure 2.3.2 Login Process Flowchart

Figure 2.3.3 Member Management Flowchart

Figure 2.3.4 Weekly Contribution Reminder flowchart

Figure 2.3.5 Context Diagram

Figure 2.3.6 data flow diagram

Figure 3.3.1 Login Code

Figure 3.3.2 Dashboard Code

Figure 3.3.3 Member's Management Code

Figure 3.3.4 Transaction Code

Figure 3.3.5 API Code

Figure 3.4.1 Login Php Code

Figure 3.4.2 Dashboard Php Code

Figure 3.4.3 Member's Php Code

Figure 3.4.4 Transaction Php Code

Figure 3.4.5 SMS Php Code

LIST OF TABLES

Table 1.4 Functional Requirements

Table 1.6 Project Schedule Breakdown

TABLE OF CONTENTS

Table of Contents

CHAPTER ONE: PROJECT PLANNING & ANALYSIS. 5

Statement of the Problem 5

Study Justification 5

System Objectives 5

General Objectives 5

Specific Objectives 5

Functional Requirements 5

Project Schedule Breakdown 7

CHAPTER TWO: DESIGN AND MODELING 7

Introduction to Modelling 7

User Interface Models 8

Login Page Design 8

Dashboard Page Design 9

Login Process Flowchart 10

Member Management Flowchart 11

Weekly Contribution Reminder Flowchart 14

Introduction 15

User Interface Development 15

Logic development 20

Testing 23

Deployment 23

Chapter Four: Conclusion And Recommendation 24

Conclusion 24

Recommendation 24

REFERENCES 25

CHAPTER ONE: PROJECT PLANNING & ANALYSIS.

Statement of the Problem

In many towns, the boda-boda groups rely on messaging platforms such as WhatsApp to communicate. The group leaders have been manually recording the weekly contributions in a notebook after the members send payments via M-Pesa. This manual system is tedious and error-prone. It also makes it difficult to track unpaid members and generate contribution reports for follow-up. A simple web-based system will be required to automate record-keeping, improve accuracy and simplify communication.

Study Justification

This project suits the identified issue well since it eliminates manual bookkeeping in favor of a simpler digital solution. This digital solution involves using a spreadsheet to automatically capture payments received and calculate totals based on the input made by the members by linking the M-Pesa transaction records together in a spreadsheet. Errors are reduced as numbers are calculated automatically, and the system makes transactions more transparent. This project is appropriate for boda-boda associations because it does not need the services of a website, technical expertise, or costly computer hardware. It only needs the use of very common hardware, such as mobile phones and computer spreadsheets.

System Objectives

General Objectives

- To create user-friendly system for tracking and managing the weekly finances of a boda-boda group.

Specific Objectives

- To design a user-friendly website for managing members' contributions.
- To automate recording of weekly contributions.
- To generate reminder messages for defaulters.
- To create a system that can produce members’ contribution report.

Functional Requirements

User	**User Activities**	**System Features**
Member	Sends the payments	M-Pesa payment recording
Group Leader	Records the payments	Automatic or manual entry
	Tracks the payments	Payment status display

	Sends the reminders to members	Identification of unpaid members
	Views the reports	Weekly contribution report

Table 1.4 Functional Requirements

1.5 Breakdown of Tools and Resources to be Used

1. Laptop

A laptop will be used for system design, spreadsheet setup and report generation.

Minimum specifications:

- Must have a good processor.
- Must have good storage.
- The operating system should be the latest.
- The RAM recommended should be 4GB or higher.

2. Front End

The front end of the system will be implemented using:

- HTML- For structuring the web pages.
- CSS- For styling and improving the user interface appearance.
- JavaScript- For system logic, payment tracking and reminder generation.

3. Integrated Development Environment (IDE)

There won't be any need for an IDE, but the following tools will be needed. They are sufficient for designing and managing the system:

- Visual Studio Code
- Notepad++
- Google chrome

4. Back End

The backend of the system entails:

- JavaScript logic
- My SQL database
- Browser Local Storage for data storage

5. Internet

An internet connection will be needed for:

- Testing the website in a browser.

- For sending WhatsApp reminder messages.
- For system hosting.

6. Other Hardware Devices

- Smartphone- for sending the WhatsApp reminders.
- Computer- for accessing the system.
- Printer- for printing the reports.

Project Schedule Breakdown

PROJECT MILESTONES						
WEEKS	**Project Planning & Analysis**	**Project Design & Modeling**	**Project Development & Testing**	**Project Deployment**	**Final Touches of System Documentation**	**Project Presentation**
5-9 Jan						
12-16 Jan						
19-23 Jan						
26-30 Jan						
2-6 Feb						
9-13 Feb						
16-20 Feb						
23-27 Feb						
2-6 Mar						
9-13 Mar						
16-20 Mar						
23-27 Mar						
2nd Apr						

Table 1.6 Project Schedule Breakdown

CHAPTER TWO: DESIGN AND MODELING

Introduction to Modelling

This chapter shows the system design and modelling of the website for the boda-boda financial management system. Before implementation, various diagrams and interface sketches were created to help visualize how the system would function and appear to users. Modelling the system helps in various ways:

- Understanding system flow clearly.
- Identifying possible design errors.
- Structuring the user interface.
- Reduce mistakes while coding.

User Interface Models

The user interface models present the interface designs that were sketched before developing the actual system. These designs were created to show how information would be displayed clearly and simply for easy understanding.

Login Page Design

This page was designed to allow the registered users to access the system securely. The page contains:

- The Email address
- Password
- Registration button for members who haven't registered.

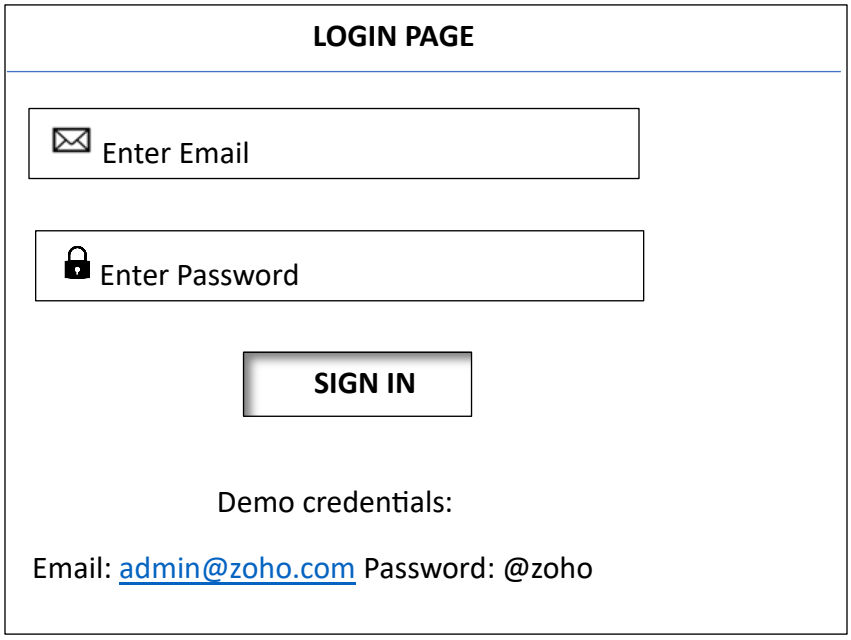


Figure 2.2.2 Login Page Design

Dashboard Page Design

This page has a sidebar that navigates to the following:

- Member Management Page
- Dashboard Page
- Transaction Page
- API Page

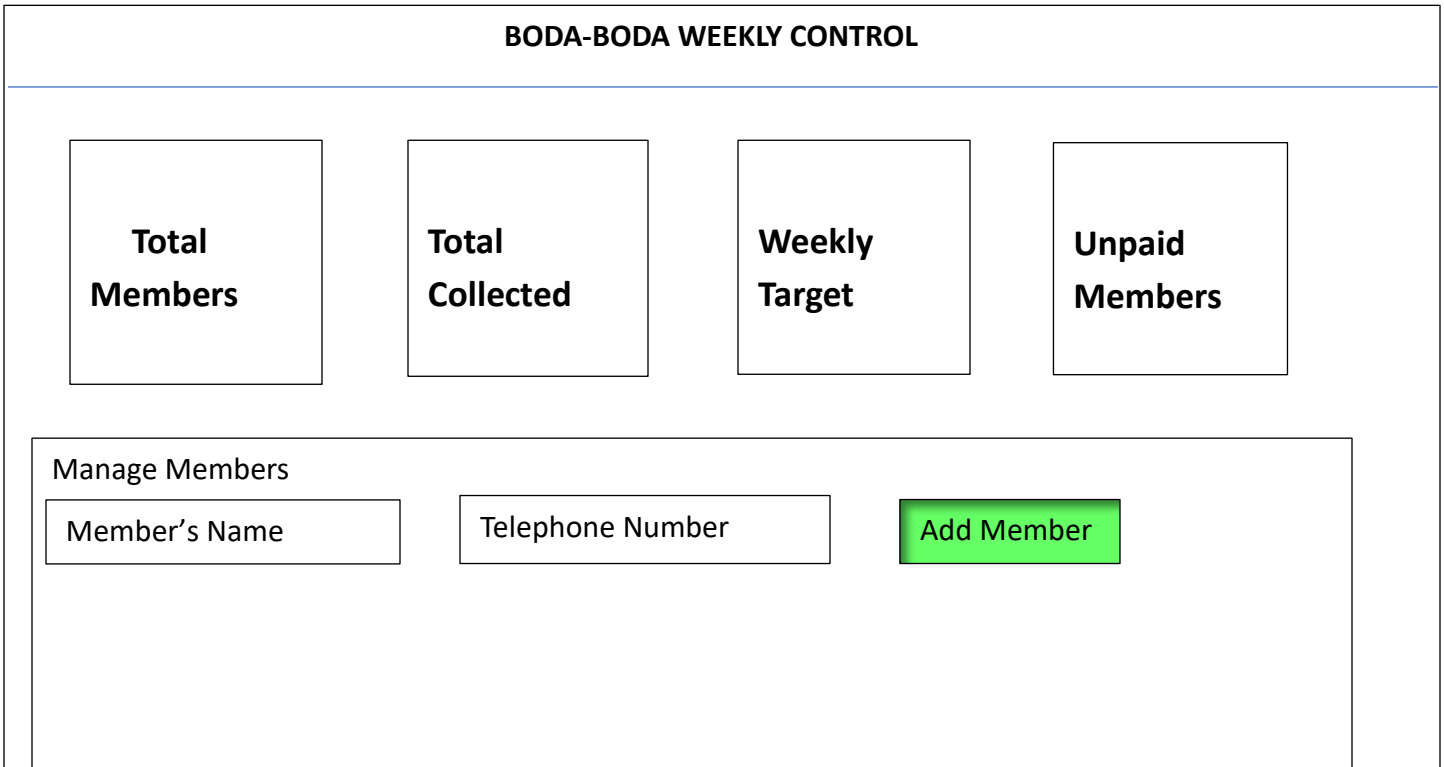


Figure 2.2.3 Dashboard Page Design

2.1 Logic Models

This section uses logic diagrams to explain how the system works internally. For example, flowcharts.

Figure 2.2.3 Dashboard Page Design

Login Process Flowchart

This flowchart shows the process for a member attempting to log in to the dashboard. The process starts when the user enters their email and password on the login page. The system will check whether the user exists in the database. If the user is not found, an error message is displayed. Access will be denied as well if the password is incorrect, if everything is correct, the member will be redirected to the dashboard.

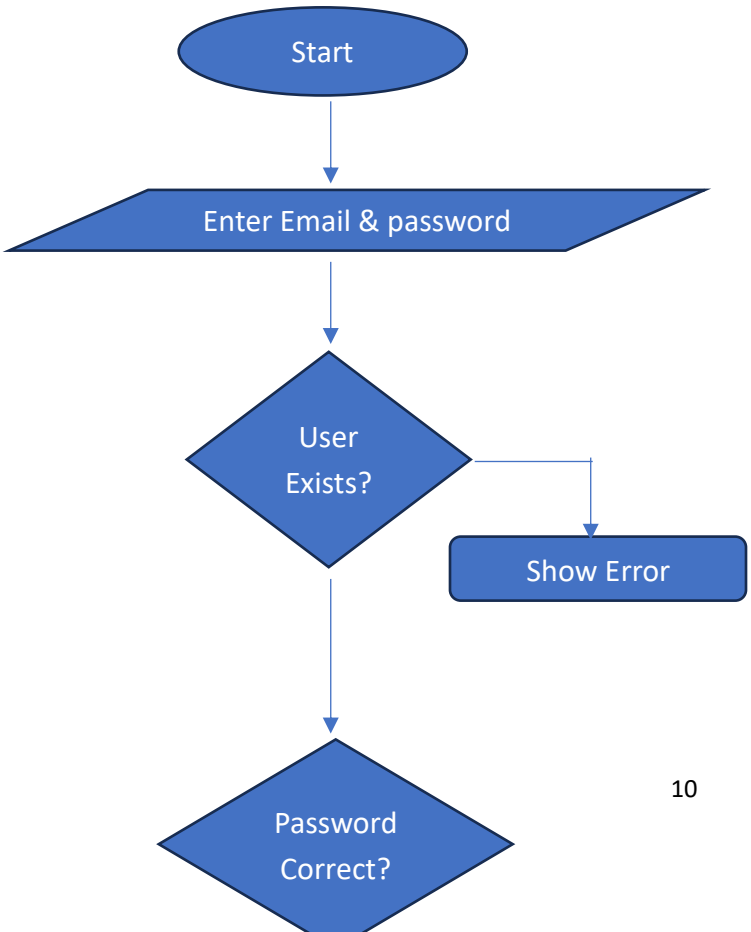


Figure 2.3.2 Login Process Flowchart

Member Management Flowchart

This flowchart shows the system's main functionality after login. The process begins by checking whether the user is logged in. Authorized personnel will be allowed to view the dashboard once it is displayed the following operations will take place:

- Loads registered members automatically.
- Calculates total collected contributions.
- Calculates weekly target amount.
- Identifies unpaid members.

The leader will perform the following actions:

- Will be able to add a member manually
- Record a payment
- Delete a member
- Manually send reminders with deadlines
- View the contribution chart
- Logout

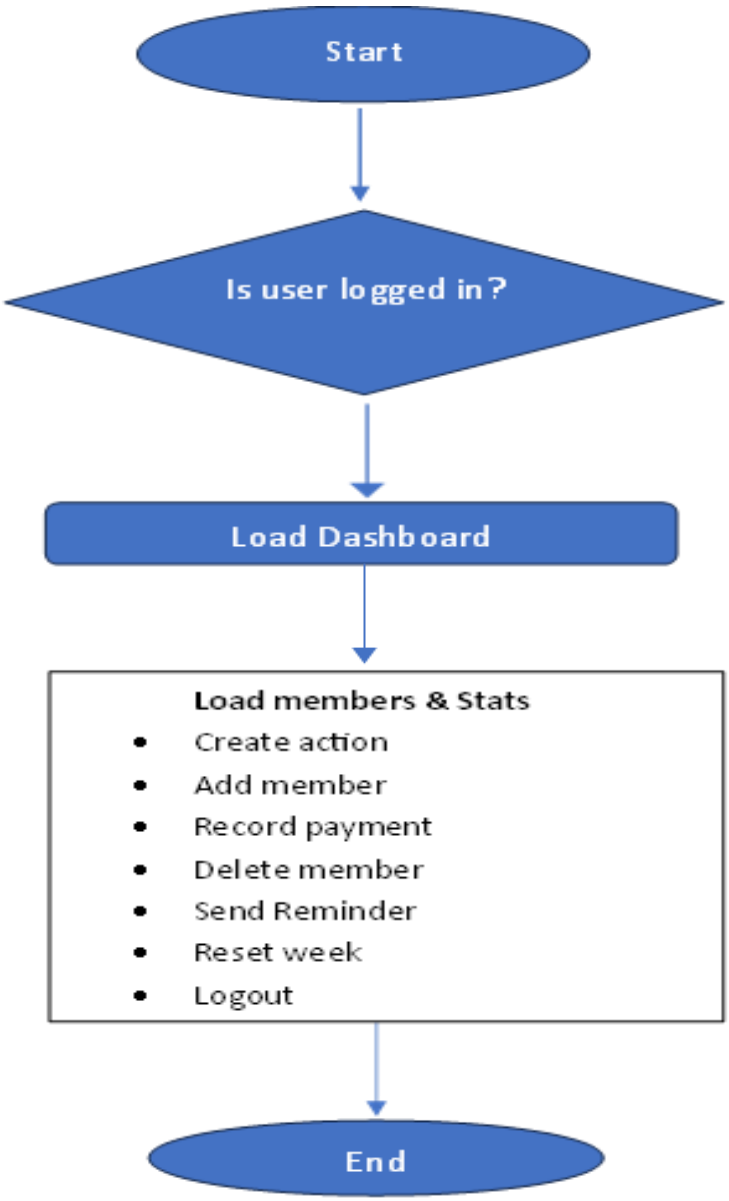
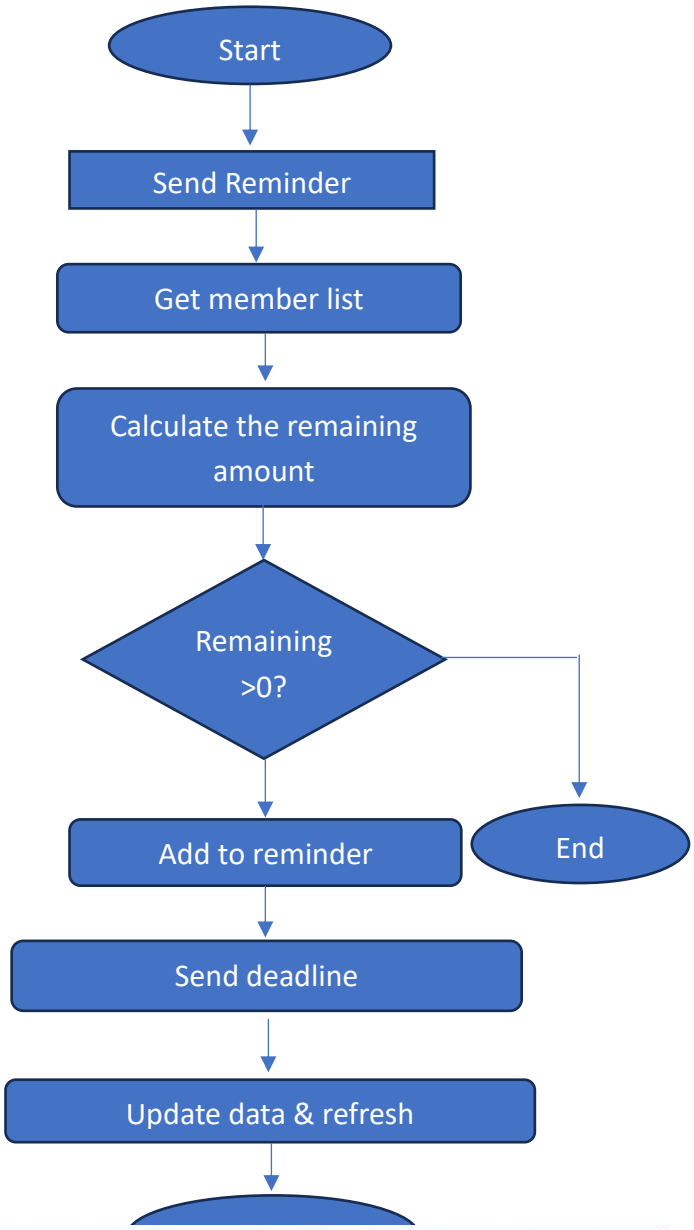


Figure 2.3.3 Member Management Flowchart

Weekly Contribution Reminder Flowchart

This flowchart shows how the system generates payment reminders for members who have not completed their weekly contributions. The leader clicks the reminder button, then the system retrieves the list of members and calculates the remaining balance for each member based on the contribution limit. The system is able to separate the members who have fully sent their contributions from those who have a debt. The system has the capability of setting a deadline and generating a reminder message.



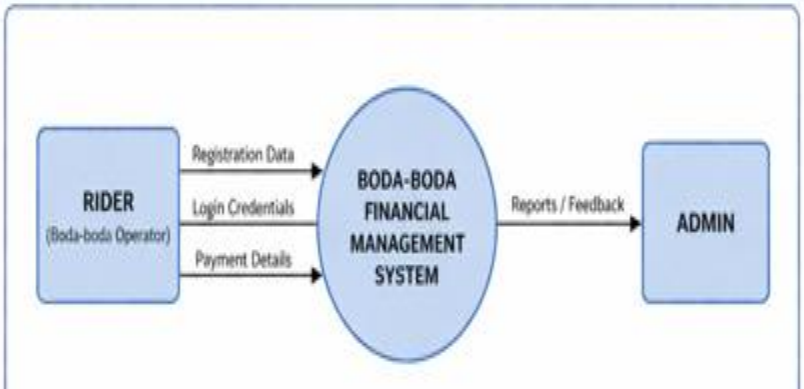


Figure 2.3.5 Context Diagram

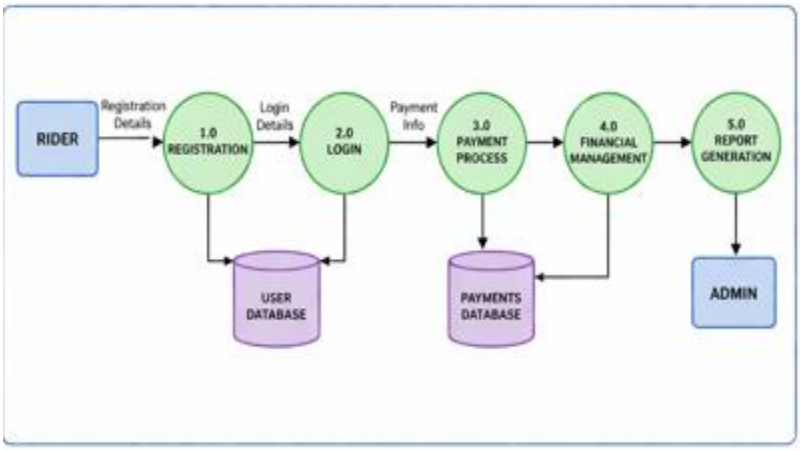


Figure 2.3.6 data flow diagram

CHAPTER THREE: SYSTEM IMPLEMENTATION

Introduction

This chapter describes the different development stages that took place, such as user interface and logic development. Some screenshots will be taken to illustrate the different development stages.

User Interface Development

Below are screenshots of the user interface.

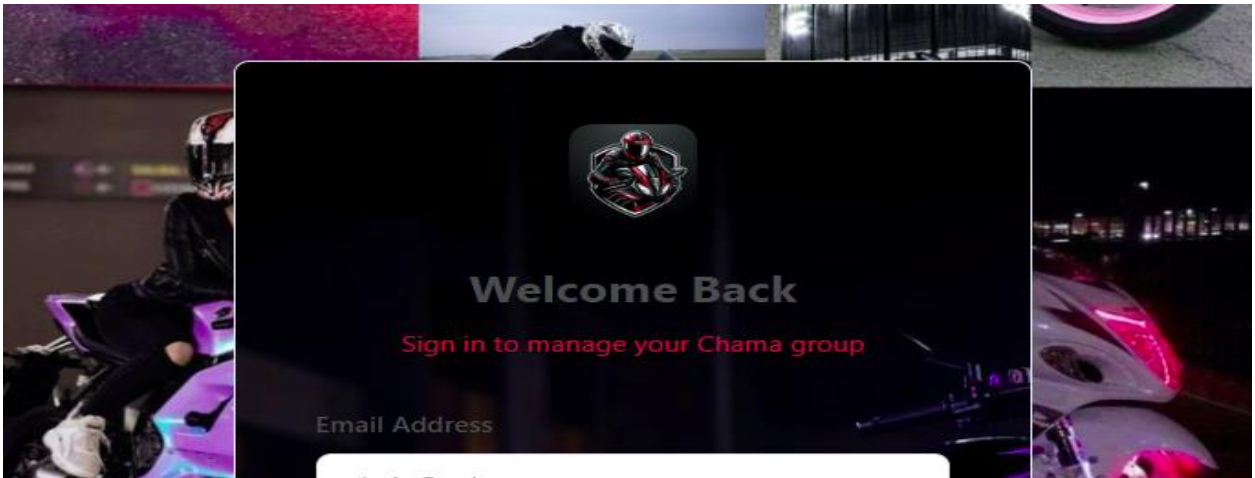
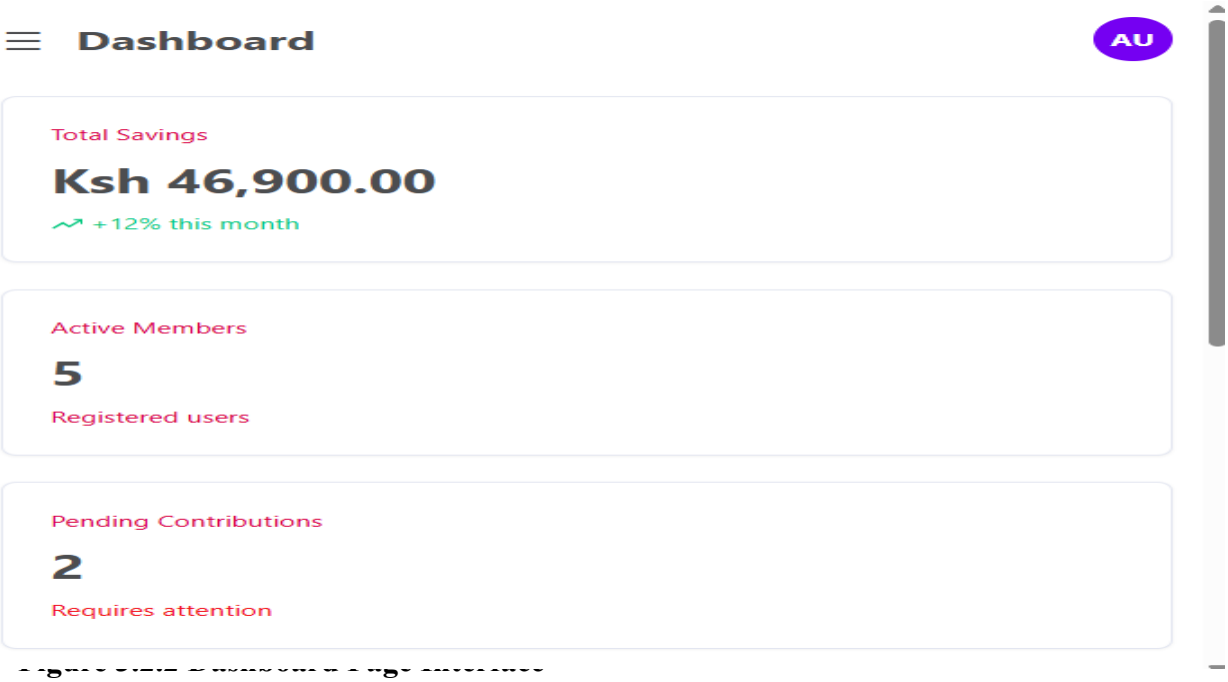


Figure 3.2.1 Login Page Interface



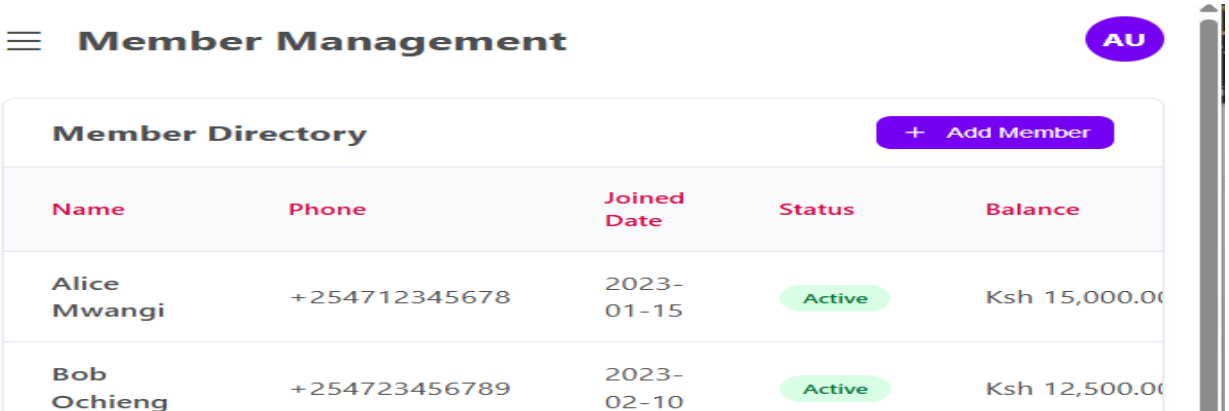


Figure 3.2.3 Member Management Page Interface

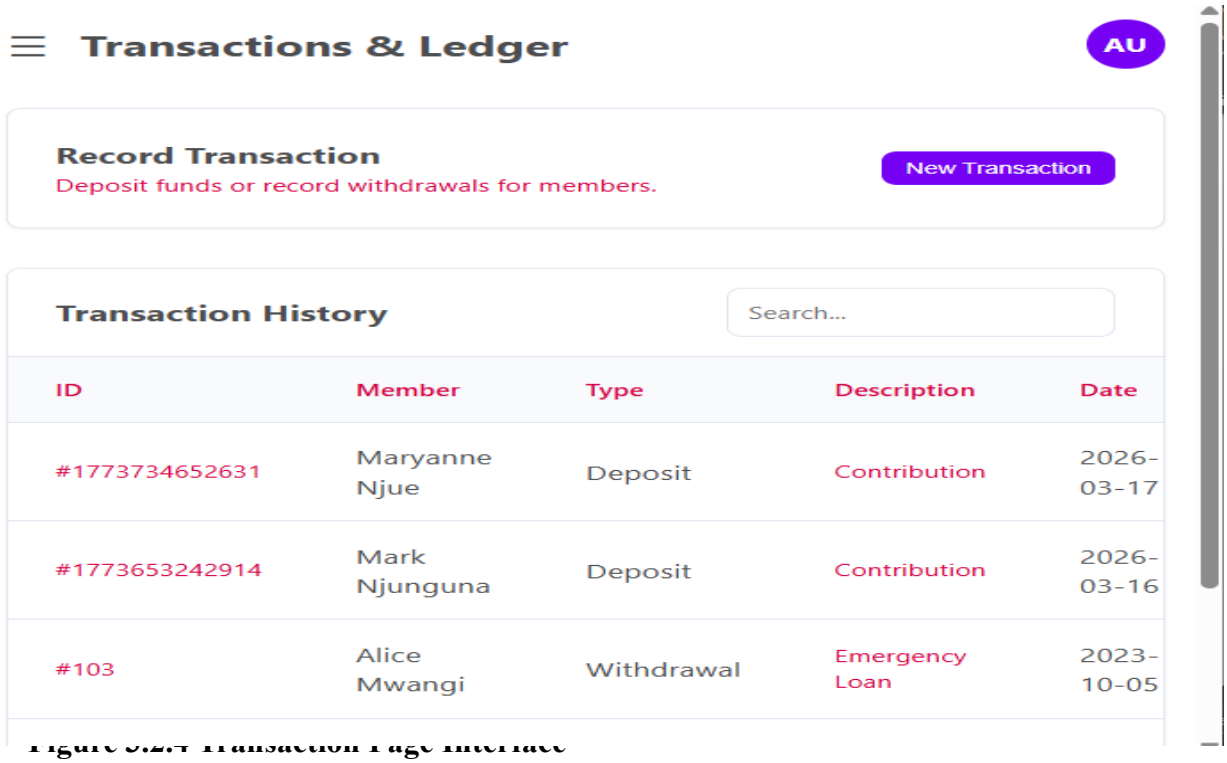


Figure 3.2.4 Transaction Page Interface

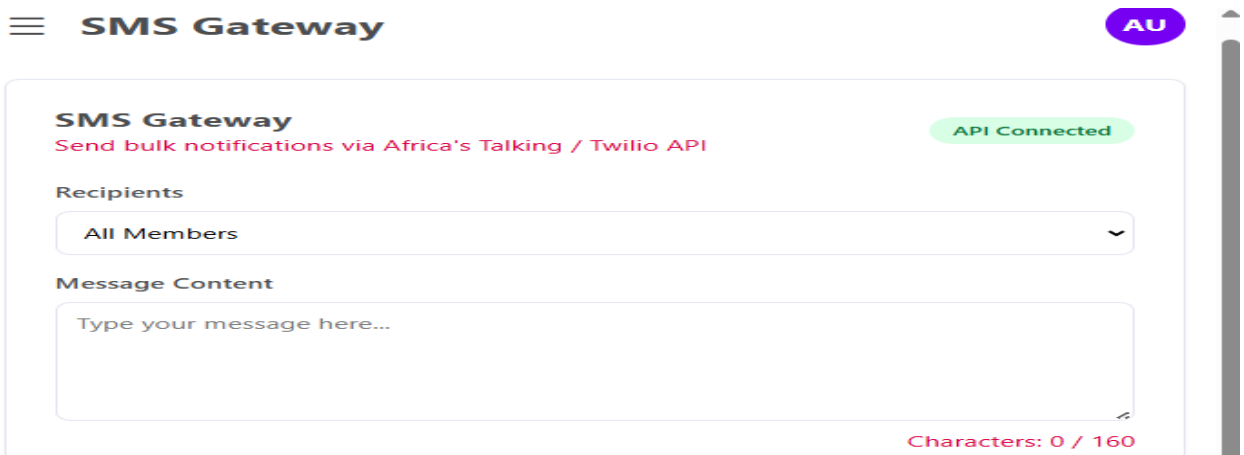


Figure 3.2.5 API page interface

```html
<!-- LOGIN SCREEN -->
<div id="login-view" class="login-container">
    <div class="login-card">
        <!-- CORRECTED LOGO SECTION: Replaced broken tag with clean SVG -->
        <div class="login-logo">
            <img src="logo.png" style="width:100%; height:100%; object-fit:contain; border-radius:12px;">
        </div>

        <h2 class="login-title">Welcome Back</h2>
        <p class="login-subtitle">Sign in to manage your Chama group</p>

        <form id="login-form" onsubmit="app.handleLogin(event)">
            <div class="form-group">
                <label class="form-label">Email Address</label>
                <input type="email" class="form-input" placeholder="admin@zoho.com" value="admin@zoho.com
            </div>
            <div class="form-group">
                <label class="form-label">Password</label>
                <input type="password" class="form-input" placeholder="••••••••" value="@zoho" required>
            </div>
            <button type="submit" class="btn btn-primary" id="login-btn">
                <span>Sign In</span>
            </button>
        </form>
        <div class="mt-4">
            <p class="text-sm text-muted">Demo credentials:<br>Email: admin@zoho.com | Pass: @zoho</p>
        </div>
    </div>
```

Figure 3.3.1 Login Code

```
268
269             <!-- View: Dashboard -->
270             <div id="view-dashboard" class="view-section">
271                 <div class="stats-grid">
272                     <div class="card">
273                         <div class="stat-label">Total Savings</div>
274                         <div class="stat-value" id="dash-total-savings">$0.00</div>
275                         <div class="stat-trend trend-up">
276                             <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
277                             <span>+12% this month</span>
278                         </div>
279                     </div>
280                     <div class="card">
281                         <div class="stat-label">Active Members</div>
282                         <div class="stat-value" id="dash-total-members">0</div>
283                         <div class="stat-trend">
284                             <span style="color: var(--text-muted);">Registered users</span>
285                         </div>
286                     </div>
287                     <div class="card">
288                         <div class="stat-label">Pending Contributions</div>
289                         <div class="stat-value" id="dash-pending">0</div>
290                         <div class="stat-trend">
291                             <span style="color: var(--danger);">Requires attention</span>
292                         </div>
```

Figure 3.3.2 Dashboard Code

```
<!-- View: Members -->
<div id="view-members" class="view-section hidden">
    <div class="table-container">
        <div class="table-header">
            <h3>Member Directory</h3>
            <button class="btn btn-primary btn-sm" onclick="app.openModal('add-member-modal')">
                <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
                Add Member
            </button>
        </div>
        <table>
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Phone</th>
                    <th>Joined Date</th>
                    <th>Status</th>
                    <th>Balance</th>
                    <th class="text-right">Actions</th>
                </tr>
            </thead>
            <tbody id="members-table-body">
                <!-- Populated by JS -->
            </tbody>
        </table>
```

Figure 3.3.3 Member's Management Code

```
    <!-- View: Transactions -->
    <div id="view-transactions" class="view-section hidden">
        <div class="card" style="margin-bottom: 2rem;">
            <div class="flex-between">
                <div>
                    <h3>Record Transaction</h3>
                    <p class="text-sm text-muted">Deposit funds or record withdrawals for members.</p>
                </div>
                <button class="btn btn-primary btn-sm" onclick="app.openModal('add-txn-modal')">New Transact
            </div>
        </div>

        <div class="table-container">
            <div class="table-header">
                <h3>Transaction History</h3>
                <input type="text" placeholder="Search..." class="form-input" style="width: 200px;" onkeyup=
            </div>
            <table>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Member</th>
                        <th>Type</th>
                        <th>Description</th>
```

Figure 3.3.4 Transaction Code

```
<!-- View: SMS Gateway -->
<div id="view-sms" class="view-section hidden">
    <div class="card" style="max-width: 800px; margin: 0 auto;">
        <div class="modal-header">
            <div>
                <h3>SMS Gateway</h3>
                <p class="text-sm text-muted">Send bulk notifications via Africa's Talking / Twilio
            </div>
            <div class="badge badge-success">API Connected</div>
        </div>

        <form id="sms-form" onsubmit="app.sendSMS(event)">
            <div class="form-group">
                <label class="form-label">Recipients</label>
                <select id="sms-recipients" class="form-select">
                    <option value="ALL">All Members</option>
                    <!-- Populated via JS -->
                </select>
            </div>

            <div class="form-group">
                <label class="form-label">Message Content</label>
                <textarea id="sms-message" class="form-input" rows="5" placeholder="Type your messag
                <div class="text-right text-sm text-muted" style="margin-top: 0.25rem;">
                    Characters: <span id="char-count">0</span> / 160
                </div>
            </div>
```

Logic development

This describes how the system processes inputs step-by-step to produce the required outputs. It shows the decision points, validations and data handling behind each major function of the system.

User Login Logic

```
<?php
include 'config.php';

if (isset($_POST['login'])) {
    $email = $_POST['email'];
    $password = $_POST['password'];

    $sql = "SELECT * FROM users WHERE email='$email' AND password='$password'";
    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        $_SESSION['email'] = $email;
        header("Location: dashboard.php");
    } else {
        echo "Invalid Email or Password";
    }
}
?>
```

Figure 3.4.1 Login Php Code

```php
<?php
include 'config.php';

if (!isset($_SESSION['email'])) {
    header("Location: login.php");
}
?>

<h2>Welcome to Dashboard</h2>
<p>You are logged in as: <?php echo $_SESSION['email']; ?></p>

<ul>
    <li><a href="sms.php">Send SMS</a></li>
    <li><a href="transactions.php">View Transactions</a></li>
    <li><a href="logout.php">Logout</a></li>
</ul>
```

Figure 3.4.2 Dashboard Php Code

```php
<?php
include 'config.php';

if (!isset($_SESSION['email'])) {
    header("Location: login.php");
}
$result = $conn->query("SELECT * FROM members ORDER BY member_id DESC");
?>
<h2>Members List</h2>
<a href="add_member.php">+ Add New Member</a>
<br><br>
<table border="1" cellpadding="10">
<tr>
    <th>ID</th>
    <th>Name</th>
    <th>Phone</th>
    <th>Email</th>
    <th>Action</th>
</tr>
<?php while($row = $result->fetch_assoc(): ?>
<tr>
    <td><?= $row['member_id'] ?></td>
    <td><?= $row['first_name']." ".$row['last_name'] ?></td>
    <td><?= $row['phone'] ?></td>
    <td><?= $row['email'] ?></td>
    <td>
        <a href="edit_member.php?id=<?= $row['member_id'] ?>">Edit</a> |
        <a href="delete_member.php?id=<?= $row['member_id'] ?>" onclick="return confirm('Delete member?')">Del
    </td>
</tr>
<?php endwhile; ?>
</table>
```

Figure 3.4.3 Member's Php Code

```php
<?php
include 'config.php';

if (!isset($_SESSION['email'])) {
    header("Location: login.php");
}

$sql = "SELECT * FROM payments ORDER BY payment_date DESC";
$result = $conn->query($sql);
?>

<h2>Transactions</h2>

<table border="1" cellpadding="10">
<tr>
    <th>Payment ID</th>
    <th>Member ID</th>
    <th>Amount Paid</th>
    <th>Date</th>
    <th>Status</th>
</tr>

<?php while($row = $result->fetch_assoc(): ?>
<tr>
    <td><?= $row['payment_id'] ?></td>
    <td><?= $row['member_id'] ?></td>
    <td><?= $row['amount_paid'] ?></td>
    <td><?= $row['payment_date'] ?></td>
    <td><?= $row['status'] ?></td>
</tr>
<?php endwhile; ?>
</table>
```

```php
<?php
include 'config.php';

require_once('AfricasTalkingGateway.php');

$username    = "YOUR_USERNAME";
$apiKey      = "YOUR_API_KEY";

if (isset($_POST['send'])) {
    $phone = $_POST['phone'];
    $message = $_POST['message'];

    try {
        $gateway = new AfricasTalkingGateway($username, $apiKey);
        $results = $gateway->sendMessage($phone, $message);
        echo "SMS Sent Successfully!";
    } catch (Exception $e) {
        echo "Error: ".$e->getMessage();
    }
}
?>

<form method="POST">
    <h2>Send SMS</h2>
    Phone: <input type="text" name="phone" required><br><br>
    Message:<br>
    <textarea name="message" required></textarea><br><br>
    <button type="submit" name="send">Send SMS</button>
</form>
```

Figure 3.4.5 SMS Php Code

Testing

While building the website, I encountered some challenges, and it took several iterations to improve it. Some of the challenges were:

- Insufficient data plan
- Power outages
- Laptop malfunctions

How they were solved:

- By making sure the data plan was a lasting one
- By working when the power is strong
- By updating the needed laptop

Deployment

Currently, the website hosting is called Infinity Free. It has security challenges, uploading and running the page in the cloud, hosting the website, and fixing the pages. The link below is the host server.

https://dash.infinityfree.com/accounts/if0_41464293/domains/motorbikesacco.great-site.net

https://maryannenjue123-ux.github.io/Motorbike-Sacco/

Chapter Four: Conclusion And Recommendation

Conclusion

The Boda-Boda Financial Management System was developed to address the challenges riders face in tracking daily earnings, expenses, and payments using manual records. The system provides a digital platform where riders can register, log in, record payments, and view financial summaries, while the administrator can monitor transactions and generate reports.

The system integrates a web interface with a backend server and a database to ensure secure storage and efficient processing of user and payment data. By incorporating mobile money payment support, the system reflects real-world transaction methods used by boda-boda operators. Testing confirmed that the system performs the core functions of registration, authentication, payment recording, data storage, and report generation effectively.

The project demonstrates how information systems can improve financial accountability, transparency, and record-keeping for small-scale transport operators. Overall, the system meets the project objectives and provides a practical solution to financial management challenges in the boda-boda sector.

Recommendation

Although the system meets its intended objectives, several improvements can enhance its performance and usability:

1. **Mobile Application Development**
   A mobile app version would make the system more accessible to riders who primarily use smartphones.
2. **Integration with Live Payment APIs**
   Full integration with live mobile money APIs (e.g., **M-Pesa**) would allow automatic payment verification instead of manual entry.
3. **Enhanced Security Features**
   Implement features such as password encryption, two-factor authentication, and secure sessions to improve data protection.
4. **Offline Capability**
   Adding offline data capture that syncs when internet is available would help riders in areas with poor connectivity.
5. **Advanced Reporting and Analytics**
   Include charts and analytics to help riders better understand income trends, expenses, and savings.
6. **User Training and Documentation**
   Provide simple user guides or tutorials to help riders who are not familiar with digital systems.
7. **Cloud Hosting Deployment**
   Deploy the system on a reliable cloud platform for better availability, scalability, and performance.

Implementing these recommendations would make the system more robust, user-friendly, and suitable for real-world deployment among boda-boda operators.

REFERENCES

Software Engineering: A Practitioner's Approach
Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Software Engineering
Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

OWASP Foundation
OWASP Foundation. (2021). *OWASP top ten web application security risks*. https://owasp.org

Node.js
Node.js Foundation. (2023). *Node.js documentation*. https://nodejs.org

MongoDB
MongoDB Inc. (2023). *MongoDB documentation*. https://www.mongodb.com/docs

draw.io (diagrams.net)
diagrams.net. (2023). *Create diagrams online*. https://www.diagrams.net

Safaricom PLC
Safaricom PLC. (2023). *M-Pesa Daraja API documentation*. https://developer.safaricom.co.ke

World Wide Web Consortium (W3C)
W3C. (2022). *HTML and CSS standards*. https://www.w3.org

Database System Concepts

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill.

Systems Analysis and Design

Kendall, K. E., & Kendall, J. E. (2019). *Systems analysis and design* (10th ed.). Pearson.